

Distributed KV Cache Storage System: Extending GPU KV Capacity with CPU/DPU DRAM Pooling for cost efficient Inference

Subtopic: Distributed-KV-Cache-Storage-System

Shengpeng DING (21185607)

ABSTRACT

KVCaches are tremendously stored on GPU for fast LLM prefilling prefix reuse and decoding acceleration. Many storage systems such as LMCache, Mooncake and HiCache are integrated in inference system to offload KVCaches for future reuse. However, most of them offload data to a low level, such as CPU pool or remote backend, and upload when data are used. This long-way vertical and cross-CPU travel of KVCaches in the storage system will take a lot of time, which makes some SLOs hard to meet. To address this, VCache¹, a VRAM-level KVCache system is presented in this article. VCache makes following contributions: (i) distributed VRAM pool with direct data transfer to enable fast in-GPU data feed. (ii) multi-level storage system enables offloading to mooncake storage backend as fallback. (iii) available system APIs such as store, retrieve and lookup.

1 INTRODUCTION

A fundamental contradiction is becoming critically acute as Large Language Models (LLMs) tackle long-context tasks—such as analyzing entire books, sustaining extended dialogues, or processing complex codebases: the collision between the model’s powerful information absorption capacity and the physical limits of GPU memory capacity and bandwidth. At the heart of this conflict lies the Key-Value (KV) Cache, whose size grows linearly with sequence length. As context lengths scale from thousands to millions of tokens, the KV Cache not only exhausts the memory of even the most advanced GPUs but also becomes the primary bottleneck for throughput and latency.

In this landscape, the traditional, coarse-grained management model of "one-cache-per-request" is no longer viable. The focus of optimization is shifting from compressing within a single request to implementing intelligent, system-level management and reuse of cache resources across requests. This evolution follows a clear, two-tiered path:

- **Prefix Caching:** This is the classic optimization for eliminating intra-request redundant computation. Its core idea is straightforward: for a given static long context (like a system prompt or a document), its corresponding KVCache remains unchanged during the entire generation process. Therefore, the system can compute and save the complete KV Cache for this prefix only once during the initial prefill phase. Throughout the subsequent Decode phase, it simply reads this cached data for each new token, avoiding the massive waste of recomputing the entire prefix repeatedly. This is commonly seen in Prefill-Decode (PD) disaggregation inference.
- **Cross-Request Shared Caching:** It is built on a critical observation: in real-world serving, a vast number of requests share identical or highly similar prefixes (e.g., the same instruction template, the same reference document, the same chatbot system prompt). The system persistently stored the computed KVCaches in memory for following requests. When a new request arrives, the system first matches its input prefix against keys in this memory. On a hit, it directly loads the pre-computed KVCache, entirely skipping the computationally heavy prefill phase.

Modern storage systems like Mooncake store [1], LMCache [2] and Hicache [3] use the second principle to build a large offloading KVCache pool. They construct a large memory layer that systematically handles the matching, storing, retrieving and transfer process, allowing multiple upper layer application’s requests to share and reuse KVCache.

However, these cache systems all make L1 VRAM and L2 DRAM fast-speed KVCache layers private to each inference instance while make L3 memory shared among all inference engines. When it’s required to retrieve previously cached KVCaches in the caching system to feed upper-layer inference applications, it takes a long way and a lot of time to transfer data from the remote instance to the local under-layer, then from the lower-layer up to the applications. Inspired by mooncake’s CPU and DRAM pooling and LMCache cache engine system architecture, this article proposed VCache, a

¹The source code is now available at <https://github.com/singserve/Distributed-KV-Cache-Storage-System>

cache system with VRAM-level memory pool which supports in-pool direct VRAM data transfer. The main contributions of this work are:

- (1) **Distributed VRAM pool with direct data transfer to enable fast in-GPU data feed:** Like each mooncake store client has DRAM segments, each VCache instance has pieces of VRAM memory in its set GPU. A large VRAM pool consists of all VRAMs scattering in GPUs, which eliminates low-speed and cross-level data loading by directly transferring in-pool KVCache to requested instance via NVLink. Centralized metadata management and distributed KVCache management like mooncake store avoid unnecessary data transfers when requesting the metadata. The distributed architecture also has good scalability.
- (2) **Multi-level storage system enables offloading to mooncake storage backend as fallback:** VCache is a 5-level cache system which has a clear vertical structure. This structure makes it possible for different modules such as segment manager and mooncake storage backend to be loosely coupled and easy to maintain. This system now utilizes mooncake store [1] as the secondary storage to pool unutilized DRAM memory among instances. This is a necessary complement for VCache since VRAM pool is not large enough to hold large amount of data.
- (3) **Available system APIs such as store, retrieve and lookup:** VCache, as a cache system, provides clear upper layer APIs for users or applications to use. Main APIs that are supported now are store(), whose role is to store given data into the system, lookup(), whose role is to lookup whether the data exists in the system given a unique key (token ids), and retrieve(), whose role is to retrieve data stored in the system given a key (token ids), whether the data are at localhost or in pool at remote instances.

2 RELATED WORK AND MOTIVATION

2.1 KVCache in LLM Inference

KVCache is a critical optimization in Large Language Model inference that stores computed attention keys and values to avoid redundant computation during autoregressive generation. During transformer inference, the attention mechanism computes key-value pairs for each token. In autoregressive generation, these KV pairs from previous tokens are needed for computing attention with new tokens. KVCache stores these pairs to avoid recomputing them, significantly improving inference efficiency.

Without any technique, when a new request comes, the inference engine will compute each pair of key-value for each

token no matter how similar this request is with the requests that come previously. Thus, computation and storage are required every time a request comes. Since 2 requests may have a long common prefix, which means the previously computed KVCache can be used this time, it is common to share KVCache across different requests.

Now most open source LLM inference frameworks have used prefix caching. vLLM [4] uses a hash-based method which splits request tokens into blocks as basic caching unit. It hashes each new block with previous block hash as if to form a chain. When new requests with the same prefix come, they will reuse the hit prefix blocks by checking the hash. SGLang [5] uses compressed prefix tree where each node can store a variable-length sequence of tokens. For a new request, the system performs a longest prefix match by traversing the tree, finding the deepest node that matches the beginning of the request's token sequence.

However, the context prefix has become longer and longer, with millions of tokens in LLM inference to generate. The prefix KVCache consequently requires a large amount storage. With this background, Mooncake [1] architecture emerges as a typical KVCache-centric disaggregated architecture with its distributed KVCache Storage by pooling under-utilized storage resources of the computing cluster, e.g. DRAM and SSD. This efficient utilization reduces the computation and storage requirements of LLM inference since the inference engine can now store computed KVCache into large DRAM, SSD spaces, etc., and retrieve them next time it needs them.

2.2 Motivation: High Delay in DRAM Transfers

Mooncake Store [1] allocates a large segment buffer in DRAM or creates files on SSD to store prefix KVCache for following requests. LMCACHE [2], known as an offloading cache system, also store KVCache into CPU DRAM and storage backends that are far from inference engine. When prefix retrieval happens at the next request, there's a long way to go for the prefix KVCache from where it is to where it's required in the GPU VRAM. This would lead to significant delay due to transfer across different type of storage resources and between CPUs. The experiment below in Figure 1 shows that the delay of cross-GPU transfer via DRAM is far longer (usually dozens of times) than that directly via NVLink bypassing DRAM. Thus, when utilizing the advantage of prefix-caching, transfer directly across GPU VRAM is preferred.

2.3 Solution: Need a High-Speed Storage and Transfer

In the previous subsection, we have seen that direct cross-VRAM transfer is much faster than cross-DRAM transfer

with 2 DRAM-VRAM copies. However, the storage of KV-Cache is on DRAM and some slow backends for both of the architectures, how to enable direct cross-VRAM transfer when retrieving prefix KV-Caches? The solution in this article is to build up a VRAM pool like CPU/DRAM/SSD pool like mooncake store [1]. The VRAM pool allocates pieces of memory segment in each GPU distributively, and manages the metadata of the KV-Cache pieces in each GPU centrally using a separate VRAM metadata manager, just like how mooncake store does using a master node. The KV-Caches are stored, managed, and retrieved within VRAM pool, which makes it fast to feed the inference applications.

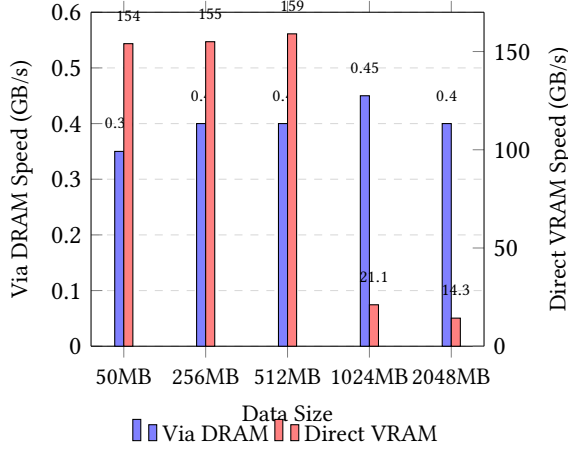


Figure 1: Data transfer speed comparison: Via DRAM (blue bars, left Y-axis) vs Direct VRAM (red bars, right Y-axis).

3 OVERVIEW OF VCACHE

3.1 VCache Architecture

VCache system is implemented as a 5-layer architecture with VCache engine to be the coordinator of almost all possible modules. Figure 2 shows the architecture of VCache system. We here briefly introduce what each layer contains and its functions and attach importance to how layers and components are coordinated to accomplish business logics in later subsection.

- (1) **Layer 1 - Application layer:** LLM inference applications like vLLM [4] run here. vLLM’s scheduler process and worker process indirectly get stored KV-Cache metadata and KV-Caches of previously handled requests through a kv connector (VCacheEngineConnector), which is an intermediate connector between vLLM processes and VCache engine.
- (2) **Layer2 – GPU VRAM Pool:** This is the true VRAM storage layer. It is placed here because all KV-Caches are

upload to vLLM applications from here and downloaded from vLLM to here, though these operations are coordinated by other controllers. This layer pooling the VRAM segments of multiple instances. Downloaded KV-Caches are stored in pool just next to the vLLM KV-Caches and direct cross-VRAM transfers happen in pool via NVLink, both of which enable fast data feed.

- (3) **Layer3 – VCache Engine:** This is the coordinator layer in the system. It incorporates upper applications and lower storage and transfer layers. It mainly coordinates components from these layers to accomplish main business workflows such as lookup, store and retrieve.
- (4) **Layer4 – Mooncake DRAM Pool:** This layer implements mooncake store [1] infrastructure to fully utilize storage resources and enable secondary DRAM pool caching as a fallback for VRAM pool. Each instance has a mooncake store clients wrapped within MooncakeStorageBackend to manage mooncake store API calls and adapt to VCache Engine’s calls.
- (5) **Layer5 – Mooncake Transfer Engine:** This layer implements mooncake transfer engine infrastructure to enable efficient cross-GPU and cross-CPU transfers. The cross-GPU transfer in memory pool is completed by a Transfer engine manager which wraps mooncake transfer engine [1] with NVLink protocol. The cross-CPU transfer is initiated by a transfer engine which is created along with mooncake store and uses TCP/RDMA protocol.
- (6) **Central Nodes:** There are 2 central nodes for VCache engine system. One is used for mooncake store metadata management and the other is VRAM pool manager which is for KV-Cache metadata management. The mooncake master [1] uses rpc calls to communicate with clients while VRAM pool manager uses ipc calls to communicate with clients now.

3.2 System Key Points

- **Chunked size KV-Cache unit:** Modern LLM inference engines such as vLLM [4] manage KV-Cache in pages, which are approximately 20KB – 63KB. The small data unit is inefficient for transferring for not saturating bandwidth. [2] VCache stores and manages VRAM unit and KV-Cache unit on DRAM backend in units of chunk with a size of configured number of tokens, which is almost the same as what LMCache [2] does. LMCache sets the default 256 tokens, which is also good for VCache since the size of a data chunk is no more than 50 MB for some popular models. As the experiment with cross-VRAM transfer shows, a considerable bandwidth speedup over

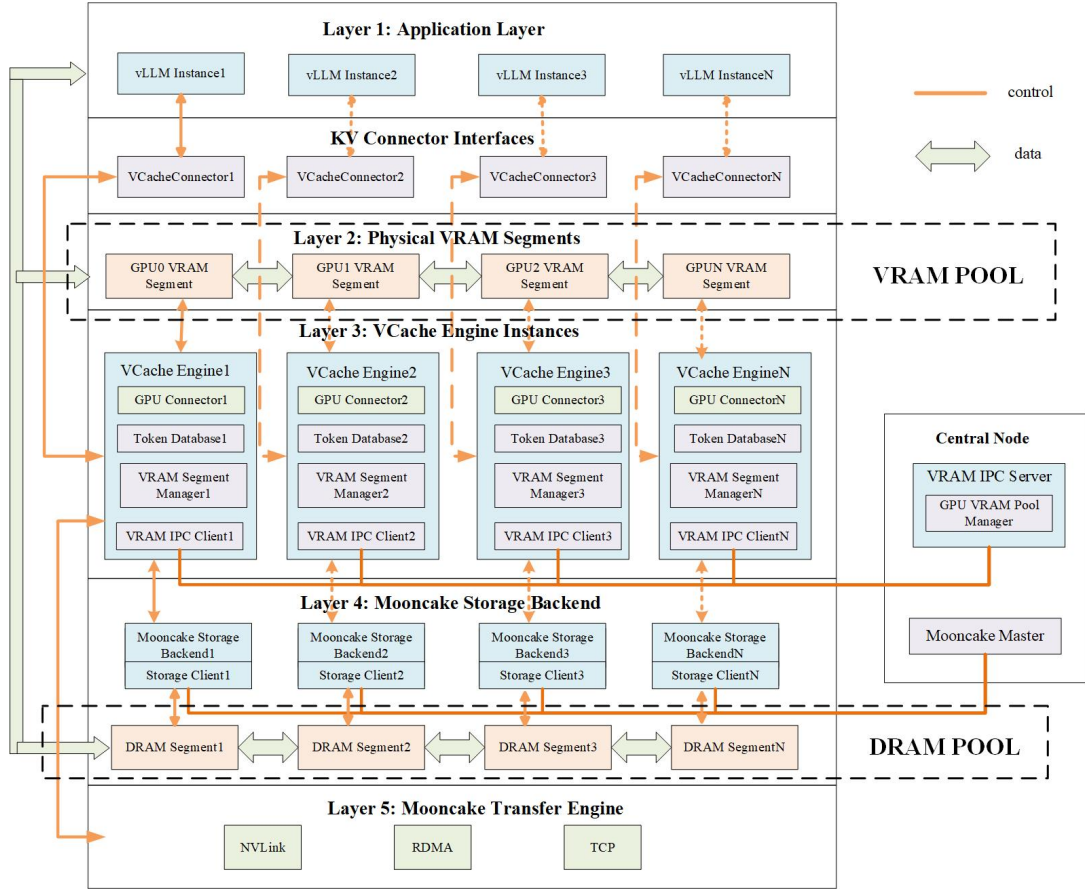


Figure 2: architecture of VCache System

DRAM transfer can be achieved when the cache size is below 50MB.

- **Batched Operations:** when several VRAM transfers are initiated, VCache combines these transfer requests and transfers them in a batch. This reduces multiple calls and initializations when it needs to transfer data blocks that are not continuously addressed.
- **Zero DRAM copy:** when KVCaches are transferred across instances in VRAM pool, the data will not be copied to DRAM for transfer. The whole process will occur in the VRAM, which is faster as is shown in the experiment.
- **Efficient VRAM management:** VRAM in each instance is allocated in large segment and managed in fine-grained blocks. VRAM blocks are managed by 2 link lists for free and allocated blocks. The VRAM management supports fine-grained free, allocated, split and merge for high utilization purpose. Each KVCache data block has only one copy in one instance, all the referenced places use data

pointers to minimize data duplication and redundant copies.

3.3 Workflows: Lookup, Store and Retrieve

- **Lookup:** The logic is illustrated in Figure 3. It is used to look up the cached prefix length given a sequence of request token ids. Lookup can be called by scheduler or worker of layer1 application processes such as vLLM [4] scheduler and worker process through the intermediate VCache connector (step 1, 2). Lookup in VCache engine first handles request token ids using TokenDatabase to split token chunks and generate unique keys (step 3). Then it requests VRAM pool manager through IPC client (step 4), the manager returns the cached prefix length in the GPU VRAM pool in the cluster by checking managed metadata (step 5). If no cached prefix in VRAM pool, then fallback to call lookup in mooncake store backend, which initiates mooncake store API calls (step 6, 7, 8, 9). After that, VCache will return the cached prefix length through kv connector (step 10). In scheduler role, VCache engine

only instantiates a light mooncake store [1] instance to execute prefix lookups. In worker role, it instantiates the whole mooncake storage backend and calls its lookup function to look up the length of cached prefix.

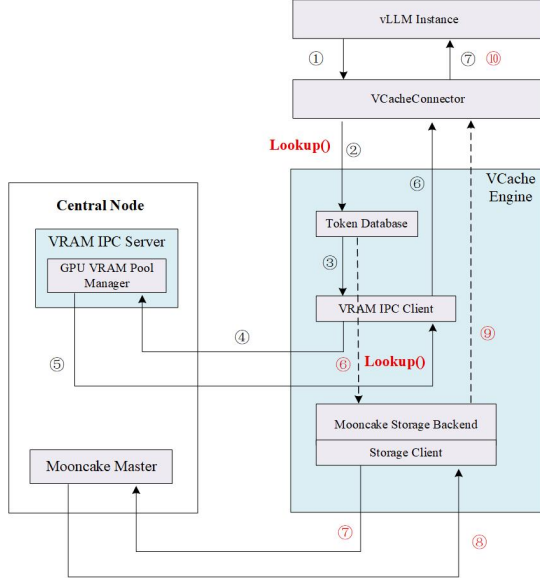


Figure 3: workflow of lookup

- Store:** The logic is illustrated in Figure 4. It is used to store the KVCache into the VRAM pool and mooncake storage backend for the following request reuse. When the store operation is called through VCache connector (step 1, 2), it first processes tokens into chunks (step 3) and checks if the KVCache which are downloaded by VCache connector from vLLM already exists in VRAM pool by making a contains IPC call (step 4, 5). Then it stores data into local GPU VRAM segment through segment manager (step 6, 7) and registers this piece of data to VRAM metadata manager through IPC client (step 8). For mooncake storage backend, VCache engine also first checks if the data exists in mooncake storage, then stores it (step 9, 10). VCache now stores KVCache both to the local VRAM and Mooncake store backend which may be a duplicate copy. An improvement to put evicted data in VRAM into mooncake storage may be done in the future.
- Retrieve:** The logic is show in Figure 5. It is used to retrieve stored prefix KVCache to the layer1 applications. When retrieve is called through VCache connector (step 1, 2), VCache engine first looks up VRAM pool by making a lookup IPC call to the VRAM metadata manager(step 4, 5). If prefix KVCache hit, manager will return where the data resides and if a transfer is required. VCache retrieves the hit data from local VRAM segment or initiates

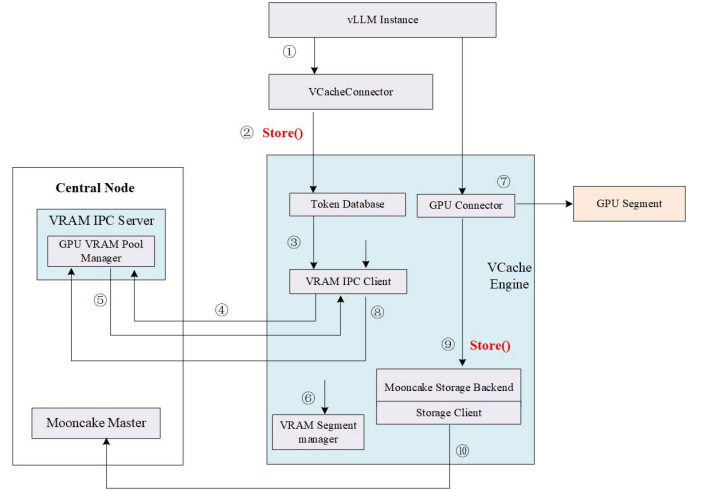


Figure 4: workflow of store

a cross-GPU transfer through transfer engine manager according to the location of hit data (step 6, 7). The remotely retrieved data are then stored locally in VRAM segment and registered through IPC to VRAM metadata manager (step 8). Whether remotely hit or locally hit, the data are now located in local GPU VRAM segment. Then, GPU connector is used to upload data to vLLM's KVCache pointers (step 9, 10). If no hits in VRAM pool, fallback to mooncake storage backend. A retrieve operation in mooncake backend is to get KVCache stored in mooncake store, which is accomplished by cross-DRAM transfer (step 9, 10).

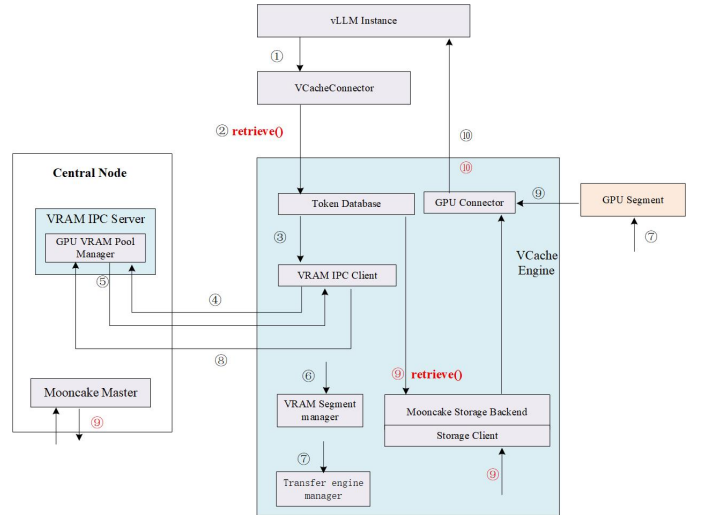


Figure 5: workflow of retrieve

3.4 Interfaces

The VCache engine is a coordinator of all modules in the system. To serve as a cache system, it provides a set of interfaces to interact with upper application layers. The interfaces and descriptions are shown in the Table 1.

4 VRAM POOL MANAGEMENT

This section will introduce the whole VRAM pool architecture, distribution of VRAM segments and centralization of metadata, as well as how memory is organized inside a segment and some memory operations.

4.1 Distributed VRAM Pool

In Distributed VRAM Pool, each VCache engine instance manages its own VRAM space using a VRAM segment manager instance. The VRAM Segment Manager instance manages all operations of its own allocated VRAM space. The space is split into several VRAM segments. Segment has its own management functions. The segment manager manages VRAM spaces by calling each segment's functions as well as its own management functions.

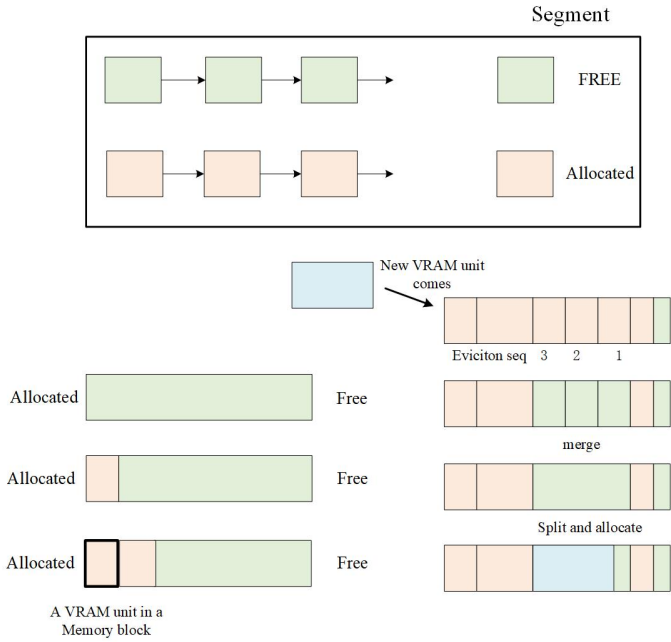


Figure 6: vram management in segment

4.1.1 VRAM Segment Manager. A VRAM segment is a large piece of VRAM buffer. Each VCache engine manages its own pieces of VRAM segment on a GPU which is accomplished by a VRAM segment manager. Each manager on a VCache engine will and must allocate a segment when VCache engine starts and segment manager is created. After

that, segments can be allocated manually if there is enough space on GPU. Segment manager manages several operations:

- **Segment data eviction:** The basic operation and storage unit is called VRAM unit, which is a piece of stored data, normally size of 256-token chunk containing model's all layers. VRAM unit only stores VRAM piece pointers, so VRAM unit only exists one copy on a GPU. A segment may contain many VRAM data units. When store operation comes, segment manager finds from all segments available space to hold the data. If no segment available, LRU protocol is used to evict VRAM units among all segments. When a VRAM unit is deleted from local VRAM, an IPC call is sent to unregister this VRAM unit. Here we just show the brief introduction of eviction, more details are in the next subsection Segment Space Management.
- **Segment allocation and free:** Segment manager can allocate segments after initialization if local GPU VRAM has enough space. But it's often executed manually. When the life of segment manager is over, it free all allocated segments.
- **Allocate space in selected segments:** Segment manager manages space allocation and free inside a selected segment. It calls the function of the segment object, which will be talked about in next subsection Segment space management. Typically the allocation and free operations are used when there is a come or go of a VRAM unit, such as eviction, retrieve, and store. After allocation and free in a segment, the manager will record some management data.
- **Manage VRAM units in local segments:** The management of VRAM units is handled also by segment manager, which is always seen along with segment space allocation and free, i.e. come or go of a vram unit.

4.1.2 Segment Space Management. VRAM Segments are large pieces of VRAM buffers. How to efficiently manage a small piece of buffer inside a segment is also of great important. The segment is split into memory blocks, each of which holds a VRAM unit. To eliminate misunderstanding, a memory block is a piece of allocated empty buffer inside a segment, and a VRAM unit is a KVCache data unit, i.e. a VRAM unit resides inside a memory block. To manage the memory blocks, a segment is divided into 2 blockchains, one connecting free blocks and the other connecting allocated blocks. Segment space management consists of several basic and simple operations, allocation, free and eviction, but merge and split operations make them a little different. The allocate, free and eviction operations are shown in the Figure 6.

Table 1: VCache engine interfaces

External APIs	Description
Lookup(tokens) -> int	Look up in the VCache system for stored KVCache, return the number of hit tokens. This function first searches the VRAM pool; if hits, it returns the hit token count. Otherwise, it falls back to mooncake backend.
Store(tokens, mask, kv caches, slot_mapping, offset) -> None	Store KV Caches with token ids into the VCache system. This function stores KV Caches in both VRAM and the mooncake store backend with slot mapping.
Retrieve(tokens, mask, kv caches, slot_mapping) -> torch.Tensor	Retrieve hit KV Caches for a sequence of tokens and upload the data to the kv caches parameter using slot mapping. This function first retrieves hit data from the VRAM pool. If no hits, it falls back to retrieve from the mooncake store backend. Returns a boolean mask indicating retrieved tokens.

- **Allocate:** Allocate in a segment for vram unit is to allocate free memory blocks. As mentioned above, free memory blocks are connected as a chain. Initially, there is only one large block in this free chain. For universality, suppose there are many free blocks as well as many allocated blocks. When a new allocation request comes, it first finds the best matched block (in size) from the free chain list, then removes it from the free list, splits it into 2 blocks, with one size of the coming vram unit and the other is the remaining. Then it returns the matched block, adds the remaining block to the free list and merges the remaining free block with the adjacent free blocks.
- **Free:** Free in a segment is to free allocated memory blocks. Allocated blocks are connected as a chain list. Initially, there is no allocated blocks. For universality, suppose there are many free blocks as well as many allocated blocks. When a free request comes, it first finds the block by the address, removes this block from allocated list, and then adds it to free list in sequence, finally merge the just freed free block with adjacent others.
- **Eviction:** Eviction occurs when there is no space in all the segments, and it's called by the segment manager. The segment manager will evict vram units using latest rarest used protocol to find the victim among all vram units in all segments. After a eviction, the freed block will be merged with adjacent free blocks to make a large block. Then manager will check if there is a large enough block

to hold the coming data. If not, it will begin next eviction. The lower right part in Figure 6 shows a example. It first evicts 3 vram units according to LRU, merges the adjacent freed blocks and allocates it to the coming data.

4.2 Centralized VRAM Metadata

This subsection describes the centralized management architecture for KVCache metadata across distributed GPU memory.

4.2.1 VRAM Metadata Server and Client. The VCache system organizes GPU VRAM storage into multiple instances distributed across different GPUs, while centralizing the management of all KVCache metadata on a single node. This architecture is implemented using a server-client model, currently operating within a single machine via Inter-Process Communication (IPC).

- **Centralized Metadata Server:** The server resides on a dedicated node and handles IPC requests from client instances by routing them to the VRAM Pool Manager. All KVCache metadata—defined as individual entries containing block location, tensor information, token IDs, and other attributes—are stored and managed centrally by this manager. The manager maintains these entries in a dictionary keyed by unique identifiers and supports core operations such as lookup, registration, unregistration, and counting. To control memory footprint, the manager enforces a maximum entry limit and employs an LRU

(Least Recently Used) eviction policy when capacity is exceeded.

- **Distributed Metadata Client:** A client component runs within each VCache engine instance on different GPUs. It acts as an interface for the local instance to interact with the central metadata service, initiating remote procedure calls for operations like metadata lookup, registration, and removal.

4.2.2 *VRAM Metadata APIs.* VRAM Metadata Management APIs and their descriptions are in the Table 2. These are used by VRAM metadata client in each VCache engine.

5 INTEGRATION WITH MOONCAKE INFRASTRUCTURE

5.1 Mooncake DRAM and Persistent Backend

The VCache engine utilizes Mooncake distributed store [1] as a DRAM-based persistent backend, primarily serving as a fallback when no hits occur in the GPU VRAM pool. This integration is encapsulated within a MooncakeStorageBackend wrapper, which interfaces with Mooncake’s API to perform KVCache lookup, storage, and retrieval operations using VCache-specific data structures. The backend is instantiated upon VCache engine creation and configured with parameters such as local_hostname, metadata_server, and protocol. Its core interfaces are summarized in Table 3.

5.2 Efficient KVCache Transfer

The Mooncake transfer engine [1] supports high-performance data transfer across CPUs, GPUs, and NVMe-oF devices via RDMA, TCP, and NVLink protocols, fulfilling VCache’s cross-device transfer requirements. VCache leverages this engine for two primary transfer scenarios:

- **Cross-GPU Transfer:** A transfer engine manager within VCache wraps a Mooncake transfer engine configured to use the **NVLink** protocol. Upon initialization, the manager registers allocated VRAM segments with the engine, enabling direct, zero-copy transfers between GPU memories.
- **Cross-CPU Transfer:** Cross-DRAM transfers occur internally within the MooncakeStorageBackend instance, which initializes a separate transfer engine using **TCP or RDMA** protocols. This engine handles data movement for storage and retrieval operations from the distributed memory pool. Currently, a single unified transfer engine supporting both NVLink and TCP/RDMA simultaneously would require extending Python APIs to expose the installTransport() method.

This architecture allows VCache to perform efficient cross-VRAM and cross-DRAM transfers by simply registering memory segments, with the Mooncake engine automatically selecting the optimal protocol.

6 STANDARD INTERFACE TO CONNECT VCACHE AND INFERENCE ENGINE

The VCache system currently integrates exclusively with the vLLM inference engine [4] using FlashAttention. This coordination is managed by a dedicated VCacheConnector, which implements vLLM’s key-value connector interface. The functions of this connector are shown in Table 4.

The VCacheConnector’s primary role is to mediate communication between vLLM’s scheduler/workers and the VCache engine, with its configuration differing based on the vLLM component it serves:

- **Scheduler Role:** Operates a lightweight instance, containing only an IPC client for VRAM pool token lookups and a minimal Mooncake store lookup client. It does not instantiate heavier modules like the storage backend, VRAM segment manager, or transfer engine.
- **Worker Role:** Instantiates all VCache sub-modules (e.g., Mooncake backend, TokenDatabase, TransferEngine manager) to perform full cache operations.

Furthermore, VCache operates in a non-layerwise mode, similar to LMCache [2]. This means all layers in model for a request are retrieved before vLLM inference begins and stored after inference completes, rather than being managed layer-by-layer during the forward pass.

7 TESTCASES

VCache engine has been tested to pass several simple functional tests with setup in Table 5.

- **Simple unit tests on VRAM segment management.** 1) test creations and basic functions of memory block such as init(), split() and merge(). 2) test creations and basic functions of VRAM segment such as allocate(), free(), get(). 3) test GPU VRAM manager, such as allocate_in_segment(), free_segment_space(), as well as some vram_unit management functions such as get(), remove(), create(). 4) test LRU evictions of VRAM data by continuously allocating segment space and filling with VRAM units. 5) test block split by allocating and freeing in batches. After freeing all memory blocks, all should be merged into one large block.

Table 2: VRAM Metadata APIs

Function Name	Description
<code>lookup_prefix(token_ids, max_tokens, current_gpu_id, all_chunks)</code>	Lookup prefix match in GPU VRAM pool metadata. If <code>all_chunks</code> is provided, find continuous matching chunks from the beginning. Only returns continuous hits from the beginning. Returns (<code>hit_tokens</code> , <code>chunk_info_list</code>) where <code>hit_tokens</code> is the number of matching tokens and <code>chunk_info_list</code> is a list of tuples for each matching chunk.
<code>register_kvcache(cache_key, token_ids, gpu_id, tensor_info...)</code>	Register a new KV cache in GPU VRAM pool metadata. This is called when vLLM generates new KV cache on any GPU or a remote transfer completes. This method only manages metadata registration, not memory allocation.
<code>Batch_register_kvcache(entries_data)</code>	Batch operation to register KVCache to GPU VRAM pool metadata. <code>Entries_data</code> is a list of the parameters in <code>register_kvcache</code> .
<code>Contains(key)</code>	Check if key exists in GPU VRAM pool metadata
<code>get_entry(key)</code>	Get metadata entry in VRAM pool manager
<code>batch_get_entry(keys)</code>	Batch operation to get entries. <code>Keys</code> is a list of <code>CacheEngineKey</code> .
<code>Remove(key)</code>	Remove metadata entry of a given key.

- **Simple store() and retrieve() on single GPU.** 1) Allocate on GPU a piece of buffer and generate mock KV-Cache with specific slot mapping. 2) create VCache engine instance. 3) store the mock KVCache into VCache by directly calling `store()` API in VCache engine. 4) retrieve just stored KVCache using the same token ids and slot mapping, but retrieve to a different address. The retrieve operation also directly calls `retrieve()` API in VCache. 5) check if the data at the 2 addresses are the same. One is what we stored and the other is what we retrieved.

- **Simple store() and retrieve() on different GPU.** 1) start store process, allocate on GPU0 a piece of buffer and generate mock KVCache. 2) create VCache engine instance with GPU0 and store the mock KVCache into VCache by directly calling `store()` API in VCache engine. 3) start retrieve process, allocate on GPU1 a piece of buffer for retrieving. 4) retrieve just stored KVCache using the same token ids and slot mapping, but retrieve the data

from GPU0 to GPU1. The retrieve operation also directly calls `retrieve()` API in VCache. 5) check if the data on the 2 GPU are the same. One is what we stored on GPU0 and the other is what we retrieved to GPU1 from GPU0.

8 DISCUSSION AND FUTURE WORK

8.1 Discussion

The core contribution of VCache, the VRAM pooling mechanism, presents inherent trade-offs. While pooling VRAM across instances as a near-application, fast data-feed layer can significantly reduce inference latency, it does so by dedicating portions of GPU memory to the cache. This allocation reduces the VRAM immediately available for model weights and computation, potentially exacerbating memory constraints. Therefore, the system essentially trades available VRAM capacity for inference speed, making it critical to carefully size the VRAM pool based on real-world workload characteristics to achieve an optimal balance.

Table 3: Mooncake backend interfaces

Function Name	Description
Store(tokens, kvcaches, kv_cache_structure, cache_key)	Store actual KVCache data to Mooncake store. Kv_cache_structure is the metadata of stored KVCache.
Retrieve(cache_key, tokens)	Retrieve KVCache data from Mooncake store using the provided cache key and token ids. This function returns hit tokens and stored KVCache with its metadata.
Lookup(tokens, all_chunks)	Lookup in the mooncake store memory pool by querying mooncake master implemented inside mooncake store. This function takes chunked token ids and corresponding keys as parameters and returns hit token number mainly.
Contains(cache_key)	Check if the cache key exists in mooncake store.

Table 4: KV Connector Functions

Function Name	Description
Scheduler side	
get_num_new_matched_tokens(request, num_computed_tokens)	Return hit tokens that can be retrieved in VCache.
update_state_after_alloc(request, num_external_tokens)	Update connector’s state after allocating blocks in vLLM.
build_connector_meta(scheduler_output)	Create connector metadata of the requests using vLLM scheduler output
Worker side	
start_load_kv(forward_context, **kwargs)	KVCache retrieve data and load all layers to kvcaches.
wait_for_layer_load(layer_name)	No operations in VCache
save_kv_layer(layer_name, kv_layer, attn_metadata, **kwargs)	No operations in VCache
wait_for_save()	Store all layer KVCache into VCache

Table 5: Testcase setup

Name	Version
lmcache	0.3.9.post3.dev3
torch	2.9.1+cu128
vllm	0.11.1rc7.dev147+gda14ae0fa.cu124
Python	3.10.12
Mooncake store	2.0.0
Transfer engine	0.3.7.post2

This system builds upon and integrates key concepts from prior work. The TokenDatabase serves a function analogous to the token processor in LMCache [2], while the KV connector and client-side lookup logic are also inspired by its design. The storage layer directly instantiates a Mooncake distributed store [1] as its backend, and the transfer engine utilizes Mooncake’s high-performance data transfer component for direct GPU-to-GPU communication. Also, the distributed memory pooling schema is also inspired by mooncake store. Furthermore, the architecture of distributed memory pooling is inspired by the Mooncake store paradigm.

8.2 Future Work

Code Refactoring and System Consolidation: While the high-level architecture and core functionality of VCache are established, the current implementation requires refactoring to improve code quality. The interdependencies between modules, functions, and data structures need to be clarified and streamlined to enhance maintainability, robustness, and to eliminate potential defects.

Performance Evaluation with Real-world Workloads:

Although the system can launch the vLLM [4] inference engine, a comprehensive performance evaluation using real-world inference traces and datasets is essential.

Support for Prefill-Decode (PD) disaggregation: Currently, VCache primarily supports a kv-both inference mode where each instance acts as both a prefiller and decoder of the cache. Supporting true PD disaggregation, where prefiller and decoder roles are separated and operate asynchronously across different instances, is a necessary step forward, as this pattern is mainstream for LLM serving

Multi-Host Distributed Deployment: Due to dependencies within the current transfer engine, VCache operations are confined to multiple GPUs within a single host. Enabling a fully distributed, multi-host version is crucial for validating the system’s scalability and for facilitating large-scale distributed inference clusters.

Extending Attention Backend Support: The current data movement layer relies on LMCache’s `transfer_multi_layers` operation, which supports specific backends like FlashAttention, FlashInfer, and MLA. To broaden applicability, future work should extend VCache to support a wider array of efficient attention computation backends.

REFERENCES

- [1] Ruoyu Qin, Zheming Li, Weiran He, Jiale Cui, Feng Ren, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. Mooncake: Trading more storage for less computation — a KVCache-centric architecture for serving LLM chatbot. In *23rd USENIX Conference on File and Storage Technologies (FAST 25)*, pages 155–170, Santa Clara, CA, February 2025. USENIX Association.
- [2] Yuhua Liu, Yihua Cheng, Jiayi Yao, Yuwei An, Xiaokun Chen, Shaoting Feng, Yuyang Huang, Samuel Shen, Rui Zhang, Kuntai Du, and Junchen Jiang. Lmcache: An efficient kv cache layer for enterprise-scale llm inference, 2025.
- [3] Liang Feng, Shikang Zheng, Jiacheng Liu, Yuqi Lin, Qinming Zhou, Peiliang Cai, Xinyu Wang, Junjie Chen, Chang Zou, Yue Ma, and Linfeng Zhang. Hicache: Training-free acceleration of diffusion models via hermite polynomial-based feature caching, 2025.
- [4] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention, 2023.
- [5] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. Sglang: Efficient execution of structured language model programs, 2024.

A TEST CASE RESULTS

The appendix shows the result of store and retrieve test case. This test case creates mock KVcache with mock slot mapping which both have the same structure with what are used in vLLM. Then the test stores the KVcache and retrieves it using another GPU. Finally, the test samples data in KVcache and compares them.

Listing 1: Test case of store and retrieve on multiple GPUs

```
1 -check if store_kv_cache and retrieve_kv_cache equals with slot mapping
2 -----
3 Store Process (GPU0): SUCCESS
4     Store time: 0.6732 seconds
5     Store KV cache address: 0x7fbf63c00200
6     Store data sample: 0.7353515625
7 Retrieve Process (GPU1): SUCCESS
8     Retrieve time: 0.4791 seconds
9     Retrieved tokens: 32/32
10    Retrieved KV cache address: 0x7ff693a00200
11    Retrieve data sample: 0.7353515625
```

The outputs below shows the GPU VRAM Metadata Manager outputs when running the above test.

Listing 2: Outputs of GPU VRAM Metadata Manager when testing store and retrieve on multiple GPUs

```
1 [2025-12-17 11:25:37,530] [LMCache INFO: Tracking 2 GPUs in VRAM pool (gpu_vram_pool_manager.
2 py:106:Lmcache.test.gpu_vram_pool_manager)
3 [2025-12-17 11:25:37,530] [LMCache INFO: Enhanced GPU VRAM Pool Manager initialized for 2
4 GPUs (gpu_vram_pool_manager.py:99:Lmcache.test.gpu_vram_pool_manager)
5 [2025-12-17 11:25:37,530] [LMCache INFO: GPU VRAM Metadata IPC Server initialized on
6 192.168.1.86:9091 (vram_metadata_ipc_server.py:84:vram_metadata_ipc_server)
7 [2025-12-17 11:25:37,530] [LMCache INFO: GPU VRAM Metadata IPC Server started on
8 192.168.1.86:9091 (vram_metadata_ipc_server.py:108:vram_metadata_ipc_server)
9 [2025-12-17 11:26:09,716] [LMCache INFO: IPC register_kvcache request - Key: b'\x94\x64\xbe\x
10 x1ak'|\x01.H\x8a\x06\xe4\xe8[\xeb\xfa\xba\x180\xfa\xadmX["48\x13Y5", GPU: 0, Tokens: 32 (
11 vram_metadata_ipc_server.py:219:vram_metadata_ipc_server)
12 [2025-12-17 11:26:09,716] [LMCache INFO: Registered GPU VRAM pool entry - Key chunk_hash: b'\
x94\x64\xbe\x1ak'|\x01.H\x8a\x06\xe4\xe8[\xeb\xfa\xba\x180\xfa\xadmX["48\x13Y5", Token IDs
: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 2,
24, 25, 26, 27, 28, 29, 30, 31], Token length: 32, GPU: 0, Segment: gpu_0_segment_0, Size:
262144 bytes (gpu_vram_pool_manager.py:336:Lmcache.test.gpu_vram_pool_manager)
[2025-12-17 11:26:09,716] [LMCache INFO: Successfully registered KV cache via IPC - Key: b'\
x94\x64\xbe\x1ak'|\x01.H\x8a\x06\xe4\xe8[\xeb\xfa\xba\x180\xfa\xadmX["48\x13Y5", GPU: 0,
Tokens: 32 (vram_metadata_ipc_server.py:240:vram_metadata_ipc_server)
[2025-12-17 11:26:12,476] [LMCache INFO: IPC batch_get_entry processed 1 keys, found 1
entries (vram_metadata_ipc_server.py:439:vram_metadata_ipc_server)
[2025-12-17 11:26:12,700] [LMCache INFO: IPC register_kvcache request - Key: b'\x94\x64\xbe\x
x1ak'|\x01.H\x8a\x06\xe4\xe8[\xeb\xfa\xba\x180\xfa\xadmX["48\x13Y5", GPU: 1, Tokens: 32 (
vram_metadata_ipc_server.py:219:vram_metadata_ipc_server)
[2025-12-17 11:26:12,700] [LMCache INFO: Registered GPU VRAM pool entry - Key chunk_hash: b'\
x94\x64\xbe\x1ak'|\x01.H\x8a\x06\xe4\xe8[\xeb\xfa\xba\x180\xfa\xadmX["48\x13Y5", Token IDs
: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 2,
24, 25, 26, 27, 28, 29, 30, 31], Token length: 32, GPU: 1, Segment: external, Size: 262144
bytes (gpu_vram_pool_manager.py:336:Lmcache.test.gpu_vram_pool_manager)
[2025-12-17 11:26:12,700] [LMCache INFO: Successfully registered KV cache via IPC - Key: b'\
x94\x64\xbe\x1ak'|\x01.H\x8a\x06\xe4\xe8[\xeb\xfa\xba\x180\xfa\xadmX["48\x13Y5", GPU: 1,
Tokens: 32 (vram_metadata_ipc_server.py:240:vram_metadata_ipc_server)
```

Minutes of the 1st Project Meeting

Date Monday, September 15, 2025
Time 9:00 am
Place Online Zoom meeting
Present Xingcheng LI
Zhihui JIANG
Shengpeng DING
Jiahua WANG
Yijun SUN
Prof. Kai CHEN
Apology None
Note-taker Yijun SUN

A APPROVAL OF MINUTES

60 minutes.

B DISCUSSION ITEMS

All students attended the kickoff meeting and confirmed the topic allocation:

- Yijun SUN introduced the four project topics and confirmed the topic allocation.

C MEETING ADJOURNMENT

The meeting adjourned at 10:00 AM. The next meeting will be held on Friday, October 3, 2025 at 9:00 AM.

Minutes of the 2nd Project Meeting

Date Friday, October 3, 2025
Time 9:00 am
Place Online Zoom meeting
Present Xingcheng LI
Zhihui JIANG
Shengpeng DING
Jiahua WANG
Yijun SUN
Prof. Kai CHEN
Apology None
Note-taker Yijun SUN

A APPROVAL OF MINUTES

60 minutes.

B DISCUSSION ITEMS

All students share their work progress and receive feedback from Yijun SUN and Prof. Kai CHEN:

- Xingcheng LI reviewed a disaggregated prefill/decode serving paper and studied Kubernetes architecture for cluster deployment.
- Zhihui JIANG surveyed vLLM and SGLang, focusing on continuous batching and PageAttention/prefix caching for memory efficiency.
- Shengpeng DING read Mooncake KV-cache papers/docs/code and summarized its master-service metadata, allocation, and get/put workflow.
- Jiahua WANG ran LMCache and Mooncake transfer-engine demos, reviewed related network optimizations, and compared their features and potential improvements.

Yijun SUN provided cluster onboarding guidance and outlined the next two-week milestones for each topic. Prof. Kai CHEN clarified course deliverables and encouraged the team to move to concrete tasks.

C MEETING ADJOURNMENT

The meeting adjourned at 10:00 AM. The next meeting will be held on Friday, October 17, 2025 at 9:00 AM.

Minutes of the 3rd Project Meeting

Date Friday, October 17, 2025
Time 9:00 am
Place Online Zoom meeting
Present Xingcheng LI
Zhihui JIANG
Shengpeng DING
Jiahua WANG
Yijun SUN
Prof. Kai CHEN
Apology None
Note-taker Yijun SUN

A APPROVAL OF MINUTES

60 minutes.

B DISCUSSION ITEMS

All students share their work progress and receive feedback from Yijun SUN and Prof. Kai CHEN:

- Xingcheng LI deployed a two-node Kubernetes cluster and brought up the vLLM production stack while debugging CUDA/driver compatibility issues.
- Zhihui JIANG deployed vLLM for online serving from source, ran a larger model on 4 GPUs, and verified it with benchmarks.
- Shengpeng DING deployed vLLM/Mooncake and continued to study Mooncake's client/control/data-plane architecture and metadata management.
- Jiahua WANG got Mooncake RDMA experiments running, compared NIXL/Mooncake/LM-Cache, and reviewed batch get/put code paths for optimization opportunities.

C MEETING ADJOURNMENT

The meeting adjourned at 10:00 AM. The next meeting will be held on Friday, October 31, 2025 at 9:00 AM.

Minutes of the 4th Project Meeting

Date Friday, October 31, 2025
Time 9:00 am
Place Online Zoom meeting
Present Xingcheng LI
Zhihui JIANG
Shengpeng DING
Jiahua WANG
Yijun SUN
Prof. Kai CHEN
Apology None
Note-taker Yijun SUN

A APPROVAL OF MINUTES

60 minutes.

B DISCUSSION ITEMS

All students share their work progress and receive feedback from Yijun SUN:

- Xingcheng LI worked on deploying Dynamo on Kubernetes, debugged multi-pod issues, and planned to deploy a model and study router/planner behavior next.
- Zhihui JIANG implemented and tested single-turn workload replay code and started extending it to multi-turn conversations with dataset/output chaining.
- Shengpeng DING wrote test code for prefix-cache hit metrics, studied vLLM metrics/logging, and refined proxy routing to target specific prefill/decode nodes.
- Jiahua WANG profiled KV-transfer overhead and fixed a local-vs-remote NIC selection bug to avoid unnecessary RDMA in local transfers.

Yijun SUN provided guidance on workload replay and debugging next steps.

C MEETING ADJOURNMENT

The meeting adjourned at 10:00 AM. The next meeting will be held on Friday, November 14, 2025 at 9:00 AM.

Minutes of the 5th Project Meeting

Date Friday, November 14, 2025
Time 9:00 am
Place Online Zoom meeting
Present Xingcheng LI
Zhihui JIANG
Shengpeng DING
Jiahua WANG
Yijun SUN
Prof. Kai CHEN
Apology None
Note-taker Yijun SUN

A APPROVAL OF MINUTES

60 minutes.

B DISCUSSION ITEMS

All students share their work progress and receive feedback from Yijun SUN and Prof. Kai CHEN:

- Xingcheng LI fixed the GPU device detection issue by installing the GPU operator and continued debugging CUDA/driver compatibility problems in Kubernetes.
- Zhihui JIANG switched the offline replay to an online setup with timestamped requests and used Grafana to evaluate performance for further tuning.
- Shengpeng DING explored GPU memory pooling integration details, resolved version mismatches, and planned tests comparing direct VRAM transfers versus DRAM-copy baselines.
- Jiahua WANG extended performance tests, built mixed all-to-all + KV-transfer scripts, and implemented an RDMA-traffic monitor for dynamic NIC selection while debugging remaining issues.

Prof. Kai CHEN noted the upcoming final presentation/report schedule and confirmed everyone received the report template.

C MEETING ADJOURNMENT

The meeting adjourned at 10:00 AM. The next meeting will be held on Friday, November 28, 2025 at 9:00 AM.

Minutes of the 6th Project Meeting

Date Friday, November 28, 2025
Time 9:00 am
Place Online Zoom meeting
Present Xingcheng LI
Zhihui JIANG
Shengpeng DING
Jiahua WANG
Yijun SUN
Prof. Kai CHEN
Apology None
Note-taker Yijun SUN

A APPROVAL OF MINUTES

60 minutes.

B DISCUSSION ITEMS

All students share their work progress and receive feedback from Yijun SUN and Prof. Kai CHEN:

- Zhihui JIANG fixed the previous latency issue and brought key metrics closer to the baseline while continuing work on CDF plots and trace generation.
- Shengpeng DING completed and demoed a test system with VRAM segment management, metadata queries, and store/retrieve transfers across GPUs, and planned vLLM/LMCache interface integration.
- Jiahua WANG reduced monitor contention by avoiding duplicated polling and started scripts to quantify baseline vs. dynamic NIC load balancing under different all-to-all scenarios.
- Xingcheng LI fixed deployment issues and successfully brought up Dynamo, then prepared router/planner configuration and workload scripts for metrics-driven scheduling analysis.

Yijun SUN coordinated the meeting order and discussed trace choices for more representative evaluations. Prof. Kai CHEN suggested keeping reports/presentations at an appropriate detail level and reiterated the final presentation/report timeline.

C MEETING ADJOURNMENT

The meeting adjourned at 10:00 AM. The next meeting will be held on Friday, December 12, 2025 at 9:00 AM.

Minutes of the 7th Project Meeting

Date Friday, December 12, 2025
Time 9:00 am
Place Online Zoom meeting
Present Xingcheng LI
Zhihui JIANG
Shengpeng DING
Jiahua WANG
Yijun SUN
Prof. Kai CHEN
Apology None
Note-taker Yijun SUN

A APPROVAL OF MINUTES

60 minutes.

B DISCUSSION ITEMS

All students share their work progress and receive feedback from Yijun SUN and Prof. Kai CHEN:

- Xingcheng LI deployed Prometheus monitoring, configured Dynamo router/planner, and prepared scripts and code analysis for final benchmarking.
- Zhihui JIANG ran experiments with ShareGPT-sampled traces under a constant-load setup and planned additional tests with shared/prefix caching traces.
- Shengpeng DING refactored the engine into worker/scheduler components, improved VRAM segment memory management, and validated store/retrieve while testing vLLM connector integration.
- Jiahua WANG adjusted experiments to create imbalanced all-to-all traffic and produced plots showing improved latency/throughput with monitor-based NIC balancing.

Yijun SUN collected weekly reports, and Prof. Kai CHEN confirmed the final presentation schedule and report deadline.

C MEETING ADJOURNMENT

The meeting adjourned at 10:00 AM. The next meeting will be held on Friday, December 19, 2025 at 9:00 AM.